

Practical 8A – Dynamic Memory Allocation

Demonstrates dynamic memory allocation using malloc(), calloc(), realloc(), and free().

```
GNU nano 8.7.1 practical8a.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    // malloc()
    a = (int *)malloc(5 * sizeof(int));

    // calloc()
    b = (int *)calloc(5, sizeof(int));

    if (a == NULL || b == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Memory allocated using malloc()\n");
    printf("Memory allocated using calloc()\n");

    // realloc()
    a = (int *)realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
        printf("Memory reallocation failed\n");
        free(b);
        return 1;
    }

    printf("Memory reallocated using realloc()\n");

    // free()
    free(a);
    free(b);
}
```

```
sri_dhruthi@Dhruthi:~$ nano practical8a.c
sri_dhruthi@Dhruthi:~$ gcc practical8a.c -o practical8a
sri_dhruthi@Dhruthi:~$ ./practical8a
Memory allocated using malloc()
Memory allocated using calloc()
Memory reallocated using realloc()
Memory freed successfully
sri_dhruthi@Dhruthi:~$ |
```

Practical 8B – Copy-on-Write

Demonstrates the Copy-on-Write mechanism using `fork()` to create a child process. When the child modifies memory, a separate copy is created while the parent's data remains unchanged.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 practical8b.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    int x = 100;  
  
    pid_t pid = fork();  
  
    if (pid < 0)  
    {  
        printf("Fork failed\n");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        printf("Child Before = %d\n", x);  
  
        x = 500;  
  
        printf("Child After = %d\n", x);  
    }  
    else  
    {  
        sleep(2);  
  
        printf("Parent = %d\n", x);  
    }  
  
    return 0;  
}
```

See 'snap info valgrind' for additional versions.

```
sri_dhruthi@Dhruthi:~$ nano practical8b.c  
sri_dhruthi@Dhruthi:~$ gcc practical8b.c -o practical8b  
sri_dhruthi@Dhruthi:~$ ./practical8b  
Child Before = 100  
Child After = 500  
Parent = 100  
sri_dhruthi@Dhruthi:~$ |
```